

Análise de Requisitos



Boas-vindas

Olá seja muito bem-vindo(a) ao curso de Análise de Requisitos! Este curso foi desenvolvido para guiar você através do cada vez mais imprescindível universo da Engenharia de Requisitos, uma área essencial para o sucesso de qualquer projeto de software.

Na primeira parte do curso, vamos explorar conceitos fundamentais, como o que são requisitos, a importância deles no ciclo de vida do software e os principais tipos de requisitos. Além disso, abordaremos os desafios da Engenharia de Requisitos e apresentaremos técnicas de elicitación, como entrevistas, workshops, questionários, observação e prototipagem, essenciais para captar as necessidades dos *stakeholders*.

Na sequência, você será direcionado(a) a métodos práticos para organizar e analisar requisitos. Vamos mergulhar nas técnicas de priorização, análise de viabilidade e impacto, oferecendo uma compreensão detalhada para assegurar que as soluções sejam alinhadas com as expectativas do projeto.

Na terceira parte, o foco será na especificação dos requisitos. Aprenderemos sobre a documentação adequada, como criar uma Especificação de Requisitos de Software (SRS) e as melhores práticas de clareza e concisão. Este é o momento de consolidar todo o conhecimento adquirido para assegurar que o projeto tenha uma base sólida.

Por fim, abordaremos a validação e o gerenciamento de requisitos, incluindo controle de mudanças e rastreabilidade. Esta última etapa é crucial para garantir que os requisitos sejam atendidos ao longo do ciclo de desenvolvimento, promovendo um alinhamento contínuo com as metas e expectativas.

Esperamos que você aproveite cada etapa deste curso, que foi pensado para fortalecer sua capacidade de análise e entendimento no desenvolvimento de sistemas. Lhe desejamos então, uma EXCELENTE jornada!

ANÁLISE DE REQUISITOS

Introdução

As atividades da vida cotidiana estão fortemente relacionadas com o uso de máquinas programáveis, seja com um computador para realizar um trabalho escolar, ou uma televisão que permite acessar diversos sistemas de *streaming* de vídeos, ou ainda os *smartphones* com um conjunto de recursos que permitem realizar desde uma chamada telefônica,

até abrir ou trancar a porta de casa remotamente e sem precisar mais de chave, e/ou realizando tal procedimento apenas com a leitura biométrica de uma digital em uma fechadura eletrônica.

Na figura 1 a seguir, podemos visualizar um exemplo de fechadura eletrônica a qual pode ser aberta/fechada remotamente via App (*wi-fi* ou *bluetooth*), por biometria e/ou senha numérica.



Figura 1: Fechadura Eletrônica

Segundo SOMMERVILLE, 2019 pp 4, “O mundo moderno não funciona sem *software*. A infraestrutura e os serviços públicos nacionais são controlados por sistemas computacionais e a maioria dos produtos elétricos inclui um computador e um *software* que o controla. A produção e distribuição industriais são completamente informatizados, assim como o sistema financeiro”. Com isso fica a pergunta: **Como desenvolvemos aplicações para tudo isso?**

Mas antes de responder, vamos conhecer um pouco sobre os tipos de aplicações e sistemas computacionais que moldam todas as áreas da vida em sociedade. Segundo Sebesta, existem algumas áreas de aplicações dos computadores e das linguagens de programação a eles associadas, para ilustrar essas aplicações o quadro 1 apresenta um resumo sobre os domínios da programação.

Quadro 1: Domínios da Programação (SEBESTA, 2018) pp 5

Aplicação	Resumo	Linguagens de Programação
Científicas	Foi responsável pela origem dos computadores nos anos 40 e 50 para aplicações com estruturas de dados relativamente simples, mas que exigiam cálculos aritméticos com ponto flutuante. Era comum o uso de vetores e matrizes, com estruturas de controle com laços de contagem e de seleções.	Fortran, Algol 60
Empresariais	Com seu início nos anos 50, eram aplicações caracterizadas por facilidades para a produção de relatórios elaborados, maneiras precisas de descrever e armazenar números e caracteres.	Cobol
Inteligência Artificial	É caracterizada pelo uso de computações simbólicas em vez de numéricas. Os símbolos compostos por nomes são manipulados. Utiliza estruturas do tipo listas ligadas de dados ao invés de vetores. Nos anos 70 surgiu a abordagem de programação lógica. Atualmente temos aplicações em Processamento de Linguagem Natural (PLN).	Lisp, Prolog, C, Scheme

Aplicação	Resumo	Linguagens de Programação
Software para a Web	São aplicações com caráter de necessidade universal para os conteúdos estáticos e dinâmicos na Web, com necessidade de alguma capacidade computacional incluída na tecnologia de apresentação dos conteúdos. Para alguns conteúdos usamos linguagens de marcação e em outros conteúdos dinâmicos usamos algumas linguagens de <i>scripting</i> .	HTML, Java, JavaScript, PHP, Perl

O panorama atual do desenvolvimento de *software* é caracterizado pela diversidade de tecnologias, metodologias e paradigmas. Esse setor é cada vez mais dinâmico, com uma evolução contínua impulsionada pela transformação digital e pela demanda crescente por soluções rápidas e escaláveis. Alguns dos principais tipo de *software* são:

- Software Comercial: Voltado para o consumidor final ou empresas, como aplicativos de produtividade, ERPs e CRMs.
- Software Livre e Open Source: Códigos abertos para colaboração e uso livre, muito utilizados por comunidades para criação conjunta e expansão de funcionalidades.
- Software Customizado: Desenvolvido sob demanda para atender necessidades específicas de empresas ou clientes.
- Software como Serviço / Software as a Service (SaaS): Aplicativos oferecidos em nuvem, permitindo acesso *online* sem necessidade de instalação local (ex.: *Google Workspace*, *Microsoft 365* e Ambientes EaD em geral).
- Aplicativos Móveis: Voltados para dispositivos móveis, como *smartphones* e *tablets*, adaptados para diferentes sistemas operacionais.
- Software para Internet of Things (IoT): Soluções que conectam dispositivos físicos à Internet, essenciais para automação e monitoramento remoto.
- Inteligência Artificial (IA) e Machine Learning (ML): Sistemas autônomos ou assistivos, cada vez mais aplicados em reconhecimento de padrões, automação de processos e análises preditivas.

Dica

Apesar de alguns trechos ficcionais, sugerimos que você assista aos filmes “**The Imitation Game**” ou em português: **O Jogo da Imitação**” de 2014 e “**Hidden Figures**” ou em português: **Estrelas Além do Tempo**” de 2016, os quais trazem muitos fatos e ideias originais sobre os primórdios dos computadores e das linguagens de programação.



Capa do filme: *The Imitation Game*



Capa do filme: *Hidden Figures*

Mais informações sobre os filmes ***The Imitation Game*** e ***Hidden Figures*** podem ser encontradas nos links do *IMDb.com, Inc* - em:

<<https://www.imdb.com/title/tt2084970/>> e

<<https://www.imdb.com/title/tt4846340/>>

Agora para responder à pergunta, **como desenvolvemos aplicações para tudo isso?** Vamos falar sobre o processo de desenvolvimento de *software* e as suas atividades relacionadas, segundo Sommerville: "Um processo de *software* é um conjunto de atividades relacionadas que levam a produção de um sistema de *software*" (SOMMERVILLE, 2019).

De forma bem prática, vamos conhecer o processo de desenvolvimento **ADIT**, que representa um processo trivial sendo o acrônimo de **Análise, Design, Implementação e Teste**.

Na figura 2, o método ADIT é representando com a indicação dos papéis (pessoas), as atividades envolvidas e da sequência para criar uma solução de *software*.

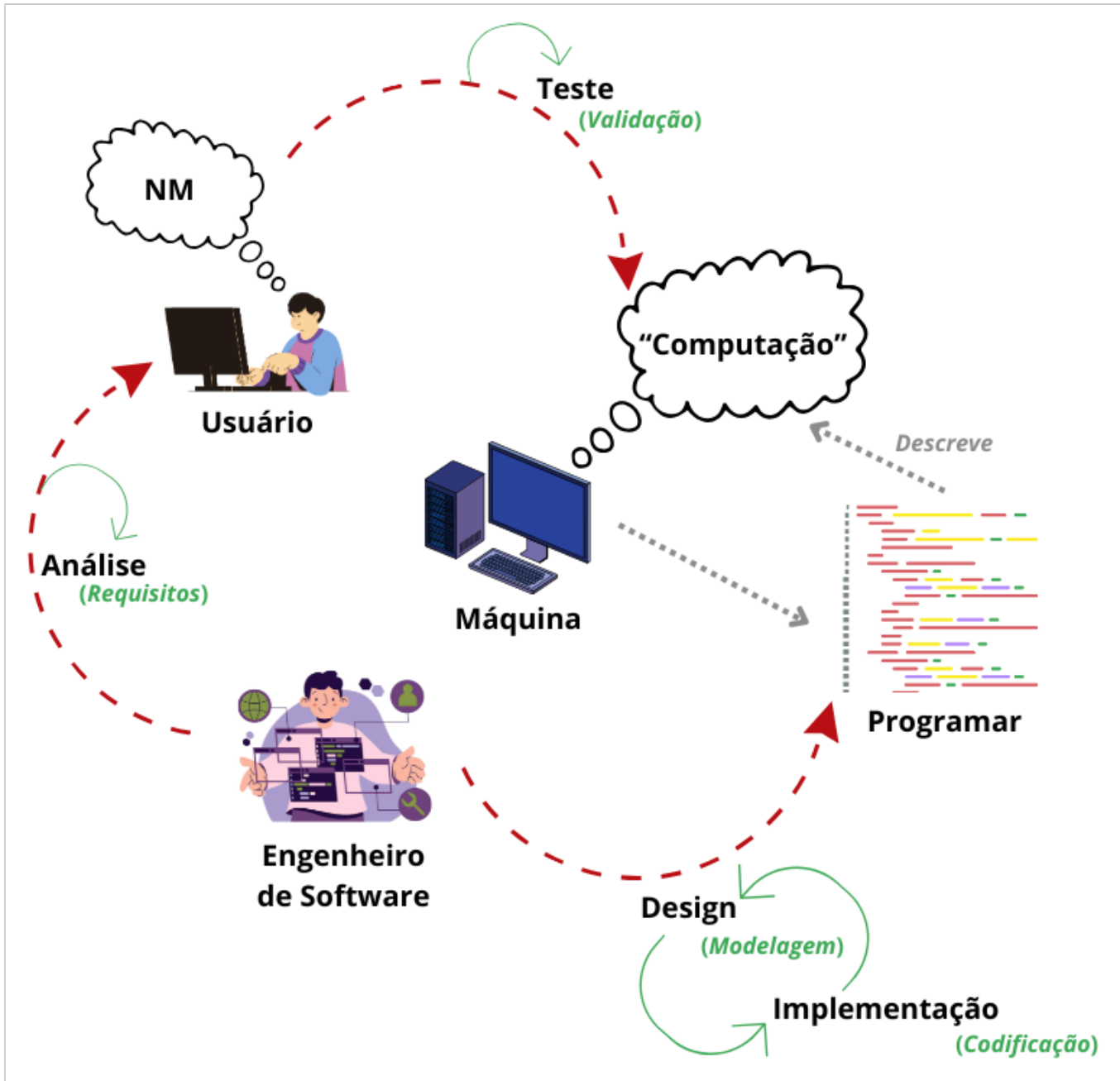


Figura 2: Processo ADIT (Análise, Design, Implementação e Teste)

Vamos entender esse processo, o Usuário tem **NM** (Necessidades e Metas) e deseja resolvê-las usando uma máquina, que é especial por ser programável, e podemos dizer que o pensamento desta é a “COMPUTAÇÃO”.

A computação representa o cálculo que a máquina realiza para satisfazer as NMs do usuário. A máquina depende de um programa para realizar a computação. E o programa descreve os cálculos que a máquina deve realizar. Portanto, um programa é a descrição dos cálculos necessários para que a máquina, ao executá-lo, realize a computação que satisfaz a NM do usuário.

Para escrever um programa, surge a figura do engenheiro de *software* que ao realizar a análise descobre os requisitos do usuário e com as atividades de design e implementação, de forma iterativa e incremental, cria a especificação do programa e implementa o código.

No quadro 2 vamos especificar as atividades que devem ser realizadas e seus respectivos atores/papeis envolvidos para desenvolver uma aplicação.

Quadro 2: Atividades e papeis do ADIT

Atividade	Papel	Descrição
Análise	Engenheiro de <i>software</i>	Descoberta dos requisitos do usuário, isto é, entendimento das (NM) necessidades e metas. Escrita dos requisitos do usuário.

Atividade	Papel	Descrição
Design	Engenheiro de <i>software</i>	Transformar os requisitos do usuário em especificação para o <i>software</i> , e realiza a descoberta dos requisitos de sistema. Realiza a modelagem do <i>software</i> podendo utilizar para tal a <i>Unified Modeling Language</i> (UML) ou Linguagem de Modelagem Unificada.
Implementação	Programador	Realiza a codificação da especificação em programa.
Teste	Testador (<i>Tester</i>)	Realiza a validação e conformidade dos requisitos.

Neste curso vamos tratar sobre como descobrir as necessidades e metas dos usuários e desta forma criar uma lista de requisitos que irão fornecer as diretrizes para a criação de uma aplicação. Aqui vamos apresentar e discutir os princípios da análise de requisitos com as práticas da engenharia de requisitos composta por: **Elicitação, Análise, Especificação e Validação**.

Para tal, a seguinte estrutura em etapas foi desenhada: **I** - Introdução a Análise de Requisitos; **II** - Técnicas de Elicitação de Requisitos; **III** - Ferramentas e Métodos para Elicitação; **IV** - Análise de Requisitos; **V** - Especificação dos Requisitos; e **VI** - Validação e Gerenciamento de Requisitos.

I - INTRODUÇÃO A ANÁLISE DE REQUISITOS

Introdução à Engenharia de Requisitos

A evolução da TI em seu meio século de existência se caracterizou pelo contínuo aumento do poder de processamento e barateamento do hardware por um lado e pela crescente complexidade do *software* pelo outro. Isto fez com que as preocupações dos responsáveis pelos sistemas de informação migrassem dos custos de hardware (decrecentes) para os custos de *software* (crescentes).

A Engenharia de Requisitos é uma área essencial no desenvolvimento de sistemas, sendo responsável por capturar, definir e gerenciar as necessidades e expectativas dos usuários e [stakeholders](#)¹. Esse processo visa assegurar que o produto final atenda aos objetivos do projeto e seja funcional, eficiente e alinhado ao contexto de uso.

Dividida em etapas como elicitação, análise, especificação e validação, a Engenharia de Requisitos envolve um conjunto de práticas e técnicas para transformar demandas em requisitos bem definidos, minimizando falhas e contribuindo para o sucesso do desenvolvimento de *software* ou de sistemas complexos. É um campo interdisciplinar que integra conhecimentos de tecnologia, negócios e psicologia, visando uma comunicação clara e precisa entre todas as partes envolvidas no projeto. (PRESSMAN; MAXIM, 2021)

O que são Requisitos?

Os requisitos de um sistema são descrições detalhadas dos serviços que ele deve fornecer e das restrições que devem ser atendidas durante sua operação. Em outras palavras, os requisitos refletem as necessidades e expectativas dos clientes e *stakeholders*, especificando o que o sistema precisa realizar para cumprir seu propósito (SOMMERVILLE, 2019, pp 85). No contexto da Engenharia de *software*, um requisito pode ser compreendido como:

- Uma condição ou capacidade necessária para que um usuário alcance um objetivo — ou para resolver um problema específico no contexto de uso;
- Uma condição que deve ser atendida por um sistema ou um de seus componentes para satisfazer contratos, padrões, especificações ou outros critérios formalmente exigidos.

Ainda em Sommerville, 2019, os requisitos devem ser claramente separados em diferentes níveis de descrição, fazendo a distinção entre os requisitos de usuário que indicam requisitos abstratos de alto nível e requisitos de sistema que indicam uma descrição detalhada do que o sistema deve fazer.

A importância dos Requisitos no Ciclo de Vida do *software*

Os requisitos são fundamentais para o sucesso de um projeto de *software* em todas as etapas de seu ciclo de vida. Sua importância pode ser destacada em diversos aspectos, conforme descritos na tabela 1, a seguir.

Tabela 1: Aspectos de um Requisito

Definição Clara do Objetivo	Os requisitos proporcionam uma compreensão detalhada do que o sistema deve realizar, alinhando as expectativas de <i>stakeholders</i> e desenvolvedores. Eles são o ponto de partida para garantir que o sistema final atenda às necessidades reais dos usuários e clientes.
Redução de Retrabalho e Custos	Definir requisitos de forma precisa e detalhada minimiza ambiguidades e reduz o risco de erros. Isso evita o retrabalho em fases posteriores, especialmente nas etapas de desenvolvimento e teste, o que se traduz em economia de tempo e custos.
Base para o Planejamento e Design	Os requisitos guiam o design do sistema, oferecendo diretrizes para a arquitetura, a modelagem e as funcionalidades. Com eles, os desenvolvedores conseguem escolher as melhores tecnologias e estratégias de implementação.
Facilitação da Comunicação e Colaboração	Os requisitos servem como uma linguagem comum entre todos os envolvidos no projeto, como desenvolvedores, analistas, clientes e gestores. Eles permitem que todos entendam o escopo e as limitações do sistema, promovendo uma colaboração mais eficaz.
Critério para Avaliação e Teste	Os requisitos são usados como base para testes e validação, garantindo que o sistema desenvolvido cumpra suas especificações. Isso permite avaliar se o <i>software</i> atende aos critérios funcionais e não funcionais estabelecidos, como: desempenho, segurança e usabilidade.
Gestão de Mudanças	Durante o ciclo de vida do <i>software</i> , é comum que necessidades e prioridades mudem. Os requisitos documentados permitem uma gestão de mudanças mais controlada, facilitando a análise de impacto e a adaptação do sistema sem comprometer a qualidade ou o orçamento.

Os requisitos fornecem um alicerce sólido para todas as fases do ciclo de vida do *software*, desde o planejamento até a entrega e a manutenção. Eles ajudam a garantir que o *software* seja desenvolvido com qualidade, atendendo às expectativas dos usuários e cumprindo os objetivos estratégicos do projeto.

Na figura 3, é apresentado um ciclo de vida de *software* com suas respectivas fases.



Figura 3: Ciclo de Vida de Software

Tipos de Requisitos

Numa abordagem tradicional os requisitos descobertos são classificados em funcionais e não funcionais. Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, os serviços que ele oferece. Já os requisitos não funcionais estabelecem limitações e condições, como desempenho, segurança e usabilidade, garantindo que o sistema funcione de forma eficiente e atenda aos padrões esperados.

Os requisitos funcionais estão associados às funcionalidades do sistema e está diretamente ligado aos resultados de comportamento do sistema. Isso significa que o que será desenvolvido no sistema resolva as necessidades que os usuários finais do sistema estão buscando.

Os requisitos funcionais dependem do tipo de sistema que vai ser desenvolvido, quem são os clientes finais e podem ser classificados em requisitos funcionais de usuário e requisitos funcionais de sistemas.

Vamos destacar as regras de negócio que estão relacionados com as premissas observadas em um negócio necessária para que ele aconteça. Os requisitos funcionais estão relacionados ao que o sistema deve fazer. As regras de negócio estão relacionadas a como o sistema deve fazer.

Os requisitos não funcionais estão dissociados das funções específicas que o sistema fornece. Na figura 4 temos um compilado dos tipos de requisitos não funcionais que são associados às propriedades do sistema.

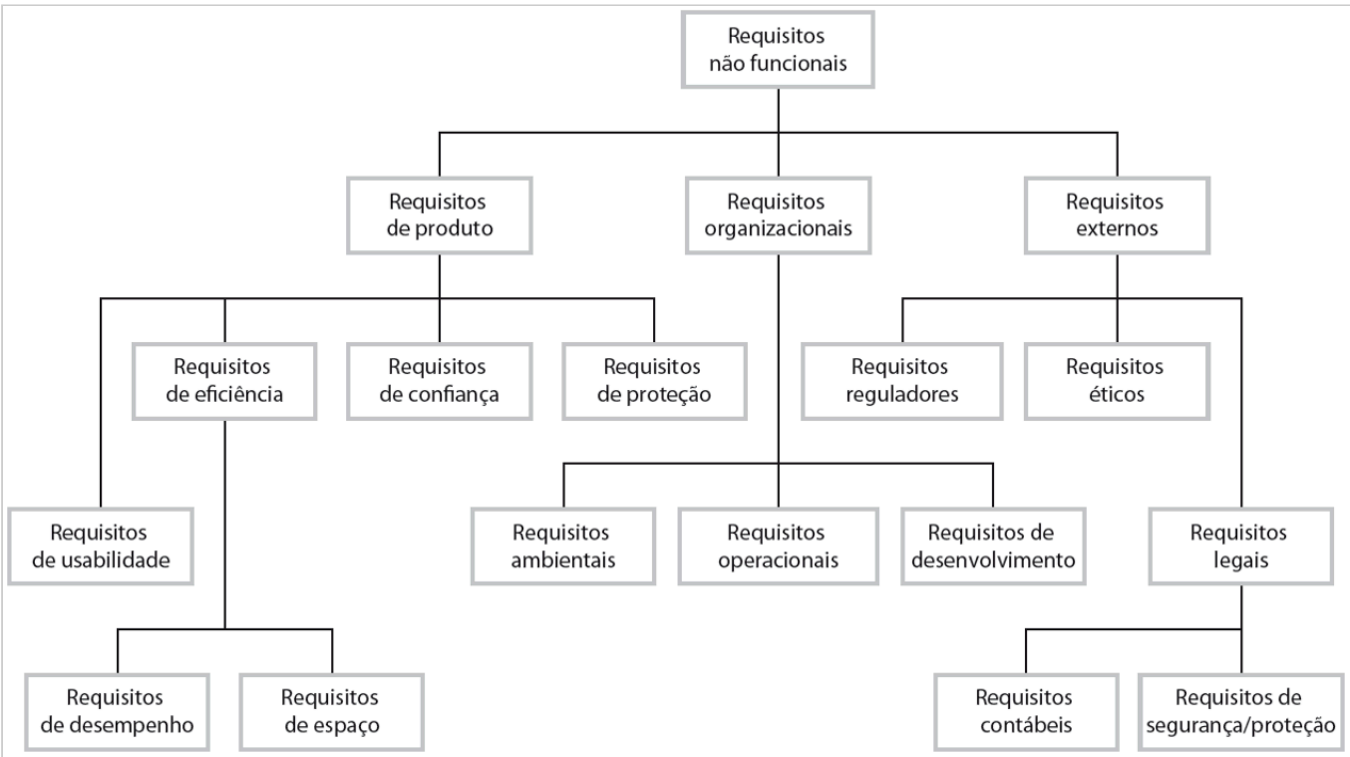


Figura 4: Tipos de Requisitos Não Funcionais (SOMMERVILLE, 2019, pp 61)

Vamos destacar os requisitos de usabilidade pois é essencial para garantir que o sistema seja intuitivo e fácil de usar. Requisitos de usabilidade bem definidos consideram diversos fatores, como:

- Facilidade de aprendizado: o sistema deve permitir que o cliente final aprenda rapidamente a utilizá-lo.
- Eficiência na conclusão de tarefas: o cliente final deve conseguir completar tarefas no sistema de forma rápida e sem dificuldades.
- Memorização da sequência de uso: o sistema deve ser intuitivo o suficiente para que o cliente se lembre facilmente da sequência de etapas necessárias para realizar uma tarefa.
- Clareza na comunicação: o sistema deve comunicar suas funções de maneira que o cliente final compreenda facilmente o que ele faz.
- Satisfação do usuário: uma porcentagem significativa dos usuários deve considerar o sistema satisfatório em termos de experiência e facilidade de uso.

Considerar esses fatores no desenvolvimento do sistema ajuda a assegurar uma experiência positiva para o usuário e contribui para o alcance das metas de usabilidade estabelecidas.

Desafio

Todos os requisitos não funcionais devem ser estudados e bem definidos, mas os imprescindíveis são: **Performance, Segurança e Confiabilidade**. Realize uma pesquisa para identificar os fatores fundamentais dos tipos de requisitos.

Principais Desafios na Engenharia de Requisitos

A Engenharia de Requisitos enfrenta vários desafios que podem comprometer a eficácia do processo e, consequentemente, o sucesso do projeto de *software*. Aqui estão alguns dos principais desafios:

- Elicitação de Requisitos: Identificar e coletar requisitos de forma completa e precisa é um desafio significativo. Os *stakeholders* nem sempre têm uma visão clara do que precisam ou podem ter dificuldade em expressar suas necessidades de maneira detalhada e compreensível. Além disso, diferentes *stakeholders* podem ter necessidades conflitantes.
- Gerenciamento de Mudanças: Durante o ciclo de desenvolvimento, mudanças nos requisitos são comuns devido a alterações nos objetivos do negócio, novas regulamentações ou avanços tecnológicos. Controlar essas mudanças e avaliar seu impacto sem comprometer o cronograma e o orçamento do projeto é uma tarefa complexa.
- Comunicação e Colaboração com Stakeholders: Muitas vezes, os *stakeholders* têm níveis variados de conhecimento técnico, o que pode dificultar a comunicação e o entendimento mútuo. Traduzir requisitos para uma linguagem técnica sem perder a essência do que o cliente espera é um grande desafio.
- Ambiguidade e Incerteza: Requisitos mal definidos ou ambíguos podem levar a interpretações errôneas, resultando em um produto final que não atende às expectativas dos usuários. Evitar a ambiguidade exige um esforço contínuo para documentar os requisitos de forma clara e detalhada.
- Priorizar Requisitos: Com frequência, é necessário lidar com uma grande quantidade de requisitos, e nem todos podem ser implementados dentro das restrições de tempo e custo do projeto. Decidir quais requisitos são mais críticos para o negócio e priorizá-los de forma a maximizar o valor do sistema é um desafio comum.
- Integração de Requisitos Não Funcionais: Requisitos não funcionais, como segurança, desempenho e usabilidade, são cruciais para a qualidade do sistema, mas podem ser difíceis de quantificar e medir. Esses requisitos, muitas vezes, são negligenciados ou especificados de maneira insuficiente, o que pode impactar a

satisfação dos usuários.

- Validação e Verificação: Garantir que os requisitos especificados refletem corretamente as necessidades dos *stakeholders* e que o sistema atende a esses requisitos durante o desenvolvimento e os testes pode ser desafiador. A validação contínua exige tempo e recursos, e a falta dela pode resultar em problemas na entrega.
- Escopo e Complexidade: Projetos de *software* podem variar muito em termos de escopo e complexidade, o que torna a Engenharia de Requisitos particularmente difícil em sistemas de larga escala. Nesses casos, gerenciar a quantidade de requisitos e assegurar sua consistência e completude é uma tarefa desafiadora.

Esses desafios ressaltam a importância de uma abordagem estruturada e colaborativa na Engenharia de Requisitos. Métodos ágeis e ferramentas de gerenciamento de requisitos, bem como práticas de comunicação eficazes, podem ajudar a mitigar esses problemas e aumentar as chances de sucesso do projeto.

¹ Podem ser usuários finais, patrocinadores "donos do negócio", gerentes, membros da equipe de desenvolvimento, clientes, reguladores, entre outros. Seu papel é essencial, pois fornecem as informações e requisitos necessários para garantir que o sistema atenda às expectativas e necessidades de todos os envolvidos.

II - TÉCNICAS DE ELICITAÇÃO DE REQUISITOS

Técnicas de Elicitação de Requisitos

A fase de elicitação de requisitos envolve o entendimento de como os *stakeholders* realizam seu trabalho atualmente e como esperam que ele seja realizado com o novo sistema implantado. Durante essa fase, o engenheiro de *software* deve descobrir as necessidades e metas dos usuários, o que inclui o entendimento do domínio da aplicação (ou seja, onde e em quais contextos essa aplicação será utilizada), as características dos serviços a serem prestados, o desempenho esperado da aplicação e as restrições de uso que devem ser respeitadas.

Existem alguns pontos de atenção no processo de elicitação. Nem sempre os *stakeholders* têm clareza sobre o que desejam do sistema; muitas vezes, utilizam termos próprios e omitem detalhes importantes de suas atividades diárias. Além disso, a visão do sistema pode variar entre *stakeholders*, e suas posições e responsabilidades podem influenciar as decisões sobre o sistema. Outro fator relevante é que os requisitos podem mudar ao longo do processo de elicitação.

Para lidar com essas dificuldades, a elicitação de requisitos deve ser um processo iterativo, trabalhando em conjunto com a fase de análise para ajustar e adaptar os requisitos conforme surgem novas informações e mudanças.

As técnicas de elicitação são utilizadas para descobrir, tornar explícito, reunir, obter o máximo de informações para o entendimento do objeto em questão. Agora vamos conhecer as seguintes técnicas: **Entrevistas, Workshops, Questionários, Observação e Prototipagem.**

Entrevista

A técnica de entrevista é uma das formas mais comuns de buscas pela elicitação de requisitos por meio de pessoas. Numa entrevista, duas pessoas conversam de forma que uma está provocando a conversa e a outra está sendo abordada.

O objetivo de uma entrevista é que o contato pessoal permita que, a pessoa abordada para a entrevista, tenha o sentimento de que faz parte do processo de resolução das necessidades do problema proposto.

As entrevistas podem ser conduzidas de forma individual ou coletiva, sendo organizadas em: **1)** Não estruturadas; **2)** Semiestruturadas; e **3)** Estruturadas. Na tabela 2 exemplificamos cada uma delas.

Tabela 2: Tipos de Entrevistas

	Não estruturada	Semiestruturada	Estruturada
Individual	“Bate-papo”	“Entrevista”	Entrevista formatada
Coletiva	Grupo focal (<i>Focus group</i>)	JAD (<i>Joint Application Development</i>) - IBM, 1977	X

A entrevista **estruturada**, também conhecida como entrevista fechada, é uma forma de entrevista em que as perguntas já foram devidamente e cuidadosamente realizadas para que a pessoa entrevistada possa respondê-las de forma direta.

Numa entrevista estruturada, as perguntas seguem um roteiro e as respostas, geralmente, são objetivas com quatro ou cinco alternativas, onde apenas uma é a correta, ou mesmo com questões de sim ou não ou verdadeiro ou falso.

Na entrevista estruturada, as perguntas são desenvolvidas pela equipe de negócios e de desenvolvimento de acordo com os requisitos levantados e que precisam ser investigados e que não permite que a pessoa entrevistada tenha condições de explicar as respostas.

A entrevista **não estruturada**, também conhecida como entrevista aberta, é uma forma de entrevista em que algumas poucas perguntas já foram devidamente e cuidadosamente realizadas para que a pessoa entrevistada possa respondê-las de forma aberta.

Numa entrevista não estruturada, as perguntas seguem um roteiro que pode ser alterado de acordo com as respostas do entrevistado. Neste formato, há poucas perguntas para elucidar requisitos com mais detalhes.

Na entrevista não estruturada, as perguntas são desenvolvidas pela equipe de negócios e de desenvolvimento de acordo com os resultados da entrevista estruturada e que precisam ser investigados com mais detalhes e que permite que a pessoa entrevistada tenha condições de explicar as respostas de forma totalmente livre e aberta.

O entrevistado, na entrevista não estruturada, tem total liberdade para responder sem interferências e o entrevistador se limita a ser um ouvinte, anotando ou gravando as respostas.

A entrevista **semiestruturada**, meio estruturada e meio não estruturada, é uma forma de entrevista em que as perguntas já foram devidamente e cuidadosamente realizadas para que a pessoa entrevistada possa respondê-las de forma aberta ou fechada.

Numa entrevista semiestruturada, as perguntas seguem um roteiro que pode ser alterado de acordo com as respostas do entrevistado. Neste formato, há um misto de perguntas objetivas e subjetivas para elucidar requisitos.

Na entrevista semiestruturada, as perguntas são desenvolvidas pela equipe de negócios e de desenvolvimento de acordo com a necessidade dos requisitos que precisam ser investigados e que permite que a pessoa entrevistada tenha condições de explicar as respostas de algumas das perguntas de forma livre e aberta.

O entrevistado, na entrevista semiestruturada, segue por perguntas abertas e fechadas com liberdade para responder sem interferências as perguntas abertas e seguir de forma objetiva com as perguntas fechadas. Nas perguntas abertas, o entrevistador fica atento às respostas passando a ser ouvinte. Na tabela 3 temos um resumo dos tipos de entrevistas.

Tabela 3: Características das Entrevistas

Estruturada	Entrevista formatada	Formal: o roteiro da entrevista é essencialmente um questionário aplicado verbalmente pelo entrevistador
Semiestruturada	Entrevista	Formal: há um roteiro básico com um <i>checklist</i> dos assuntos a serem cobertos
Não Estruturada	Bate-papo	Informal: permite descobrir aspectos culturais e políticos mais facilmente

Workshops

O workshop de requisitos é uma técnica de eliciação em grupo realizada em uma reunião estruturada, que reúne uma equipe de analistas e uma seleção de *stakeholders* que melhor representam a organização e o contexto de uso do sistema (“Técnicas para levantamento de Requisitos”, [s.d.]). Essa técnica busca obter um conjunto de requisitos bem definidos, promovendo uma compreensão coletiva sobre as necessidades e restrições do sistema.

Diferente das reuniões convencionais, onde a interação entre os participantes pode ser limitada, o workshop incentiva o trabalho em equipe e a colaboração ativa. Para facilitar a discussão e assegurar que todos os participantes contribuam, um facilitador neutro é designado para conduzir o workshop. Esse facilitador promove o diálogo entre os participantes, garantindo que todos os pontos de vista sejam considerados e que as decisões sejam tomadas de maneira colaborativa e baseada em processos estruturados de negociação.

Uma técnica frequentemente usada em workshops é o *brainstorming*, que permite explorar muitas ideias em pouco tempo e facilita a identificação de requisitos iniciais e de novas possibilidades. Após o término do workshop, é essencial produzir documentações detalhadas que reflitam os requisitos discutidos e as decisões tomadas para orientar o desenvolvimento do sistema.

Alguns aspectos importantes a serem considerados para o sucesso do workshop incluem:

- Papel do facilitador: o facilitador deve agir como mediador e observador, garantindo a imparcialidade e promovendo a participação ativa de todos.
- Planejamento e logística: a convocação deve especificar claramente o dia, horário, local e a duração do workshop, bem como o tema a ser discutido.
- Documentação: ao final, é fundamental produzir um documento que registre os requisitos levantados e as deliberações realizadas, servindo como referência para etapas futuras do projeto.

Essa abordagem permite uma visão mais ampla e participativa dos requisitos, aumentando as chances de que o sistema final atenda às expectativas dos *stakeholders* e minimize retrabalhos.

Questionários

O questionário, também conhecido como pesquisa de opinião, é uma ferramenta de eliciação que busca obter informação sobre o negócio e sobre a interação com o usuário final, porém, que estejam distantes fisicamente ou pela necessidade de um grande volume de informações.

O questionário pode ser composto por perguntas abertas ou fechadas e que tenham sido construídas com cuidados que seguem uma sequência lógica e sequencial. Um questionário é frio e impessoal e os resultados não são tão ricos.

Porém, os resultados de um questionário, geralmente, são utilizados para tratamentos estatísticos. Até porque o cuidado que se tem ao realizar as perguntas de um questionário considera perguntas curtas e rápidas, sem direcionar as respostas, mas que possuam respostas que o analista de requisitos realmente precisa.

Observação

A observação é uma ferramenta de elicitação que o analista de requisitos utiliza com aplicação em campo, para entender de perto mais sobre o negócio e as necessidades que precisam ser resolvidas.

Com a observação, o objetivo está além de apenas escutar e olhar, mas em conseguir perceber situações específicas que são fundamentais e precisam ser considerados para a assertividade do negócio.

A observação tem o objetivo de ser realizada sem qualquer interferência no ambiente, porém, a presença de uma pessoa que está dissociada do ambiente natural, acaba criando uma certa curiosidade ou desconforto.

A ferramenta de observação acontece pelo período necessário até que o observador tenha alcançado os objetivos da observação, realizando as anotações pertinentes. Na tabela 4 detalha o que se vê em cada tipo de observação a ser realizada.

Tabela 4: Tipos de Observações

Tipos de Observações	O que se vê
Distanciada	O observador apenas “olha”.
Participante	O observador entra no próprio processo a ser investigado, assumindo um dos papéis envolvidos.
Consentida	Os “observados” sabem que estão sendo observados.
Anônima	A observação é realizada sem que as pessoas que estão sendo observadas o saibam.

Na técnica de Observação podemos definir papéis de atuação do analista de requisitos, mesclando os tipos de observações com a forma de atuar, na tabela 5 vamos ilustrar os papéis com a descrição das atividades realizadas e seus impactos.

Tabela 5: Papeis na Observação

Papel	Descrição
-------	-----------

Árvore	Na observação anônima distanciada (“árvore”), o observador faz a observação sem participar do processo e sem o conhecimento dos observados. A impossibilidade de fazer perguntas leva esta modalidade de observação a ser indicada apenas quando o analista já conhecer algo do processo. Por não inibir os observados, é uma técnica especialmente indicada para procurar ver o processo como ele realmente é, ou seja, como ele ocorre no dia a dia, o que muitas vezes é muito diferente do processo documentado.
Colega	Nesta modalidade, o analista assume um dos papéis do processo, mas sem o conhecimento dos demais participantes, que tendem a vê-lo como um “colega”. Permite adquirir conhecimento tácito às vezes impossível de ser adquirido apenas com a observação distanciada. “Observador”: Essa é a situação mais conhecida: o analista observa o trabalho enquanto é executado, com o conhecimento das pessoas. Caso o processo observado seja muito complexo, aspectos de conhecimento tácito podem ficar de fora pelo fato de o analista não participar nele.
Consultor	Neste caso, o observador participa do processo com o conhecimento dos observados, assumindo um papel semelhante ao de um consultor. A participação leva ao entendimento de elementos não verbalizados pelos usuários em entrevistas ou impossíveis de observar de forma distanciada.

Prototipagem
.....

A prototipação é uma técnica que visa explorar aspectos críticos dos requisitos de um produto, permitindo a implementação rápida de um pequeno subconjunto de funcionalidades. O protótipo é particularmente útil para avaliar alternativas de interface do usuário, identificar problemas de comunicação com outros sistemas e verificar a viabilidade de requisitos de desempenho.

Diversas técnicas podem ser aplicadas na elaboração de protótipos, como criação de interfaces de usuário simplificadas, relatórios textuais e gráficos, entre outras formas de representação visual e funcional. (“Técnicas para levantamento de Requisitos”, [s.d.])

Entre os principais benefícios da prototipação estão a redução de riscos na construção do sistema, pois o protótipo permite que usuários-chave avaliem as interpretações iniciais dos requisitos feitas pelo analista. Essa validação precoce ajuda a corrigir falhas de entendimento antes do desenvolvimento completo, minimizando retrabalho.

Para que a prototipação seja eficaz, alguns fatores são essenciais:

- Escolha do ambiente de prototipagem: selecionar ferramentas e tecnologias adequadas para rápida construção e modificação do protótipo.
- Clareza nos objetivos: todos os stakeholders devem entender o propósito do protótipo, como testar hipóteses ou validar funcionalidades específicas.
- Foco nas áreas menos compreendidas: priorizar aspectos do sistema que apresentam maior incerteza ou complexidade.
- Rapidez na construção: o protótipo deve ser desenvolvido rapidamente para agilizar o feedback e permitir ajustes iterativos.

Essa técnica ajuda a alinhar as expectativas dos stakeholders, promovendo uma visão mais precisa do sistema a ser desenvolvido e contribuindo para um processo de desenvolvimento mais eficiente e direcionado.

III - FERRAMENTAS E MÉTODOS PARA ELICITAÇÃO

Modelo de Domínio

Um modelo de domínio é uma abstração dos elementos relevantes para a solução de um problema específico, permitindo representar, de forma organizada, os conceitos e relações do domínio do negócio. O propósito da modelagem de domínio é criar um glossário comum que facilite a comunicação entre todos os envolvidos no desenvolvimento do sistema, garantindo que haja um entendimento compartilhado sobre os termos e conceitos do projeto.

No processo ICONIX ("Iconix", [s.d.]) de desenvolvimento de sistemas tem a seguinte recomendação de fases para se construir um modelo de domínio.

- Primeiramente encontrar os conceitos que representam as abstrações do domínio do problema;
- Eliminar conceitos ambíguos, incorretos e itens desnecessários, isto é, eliminar os erros de abstração;
- Fazer generalização, se necessário; e
- Revisar o diagrama construído, analisando principalmente as associações e generalizações.

O modelo de domínio é geralmente construído usando o diagrama de classes da UML. Cada classe no diagrama representa um termo ou conceito central do domínio do negócio em análise, refletindo os elementos fundamentais que devem ser considerados na solução do problema. Essa modelagem não apenas facilita a definição do glossário de termos, mas também serve como uma referência importante para a equipe, auxiliando na compreensão e consistência do sistema a ser desenvolvido.

A figura 5 ilustra os elementos básicos de um diagrama de classes.



Figura 5: Elementos do Diagrama de Classes da UML

O diagrama de classes é composto pelas classes, representadas pelos retângulos com o nome da classe iniciando com letra maiúscula e sempre teremos uma ligação entre as classes, na figura 6 são representados os tipos de ligações entre as classes.

O modelo de domínio é fundamental para auxiliar a escrita do glossário de termos do projeto. Pois cria-se um vocabulário comum ao projeto e a todos os envolvidos no desenvolvimento do sistema.

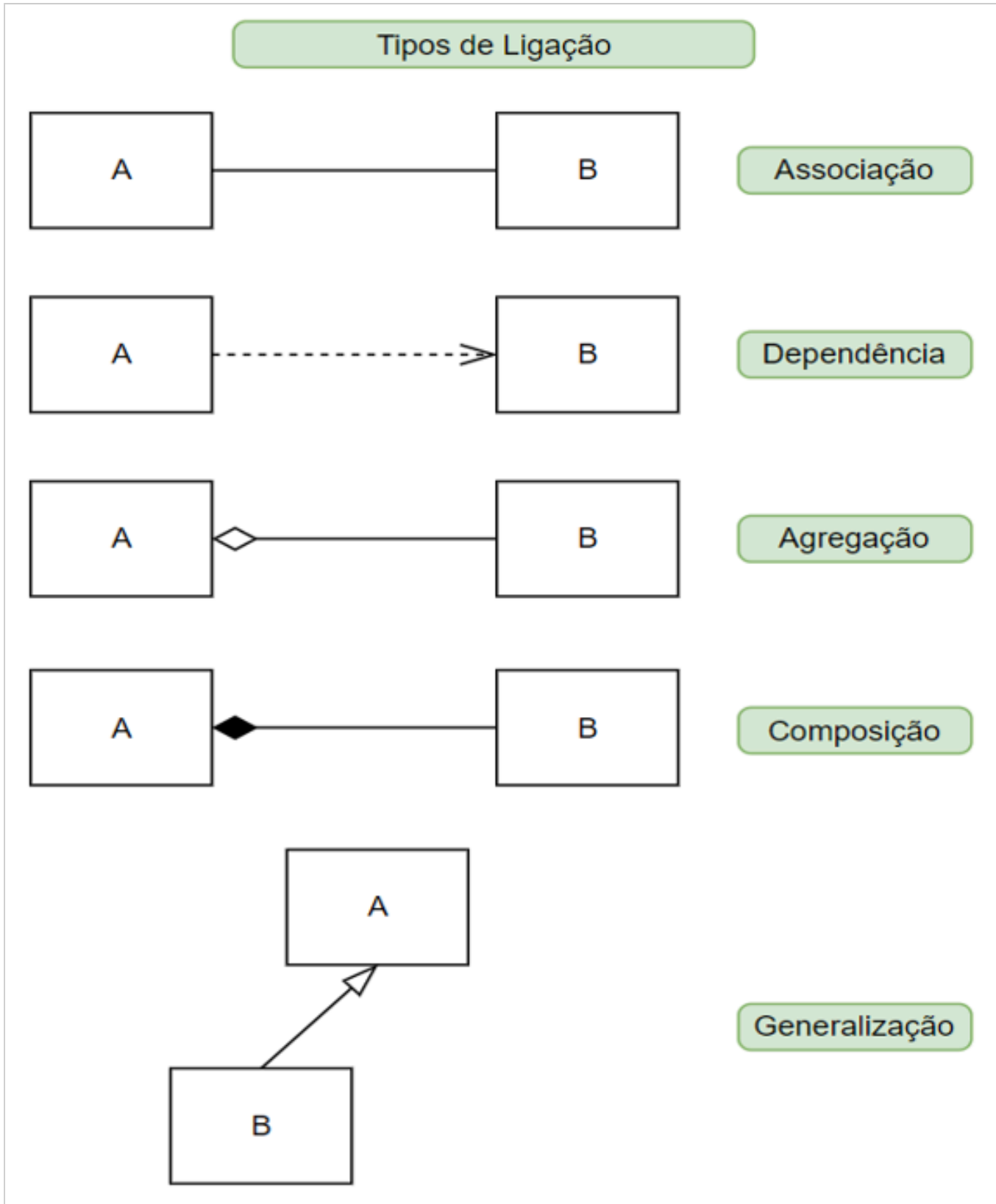


Figura 6: Tipos de ligações entre as classes

Na tabela 6 vamos descrever as ligações e indicar um exemplo baseado num sistema de gerenciamento de Fornecedores.

Tabela 6: Descrições das ligações entre as classes

Ligação	Descrição	Exemplo
Associação	Representa um vínculo simples entre duas classes, indicando que uma classe utiliza ou está relacionada à outra de alguma forma. A associação pode ter uma cardinalidade que indica quantas instâncias de uma classe podem estar associadas a instâncias de outra.	No Sistema de Gerenciamento de Fornecedores, a relação entre Fornecedor e Produto é uma associação, onde um fornecedor pode fornecer vários produtos

Ligação	Descrição	Exemplo
Dependência	Uma relação mais fraca onde uma classe "usa" outra temporariamente, geralmente indicada com uma seta tracejada. Indica que uma classe depende de outra para certas operações, mas não precisa manter uma associação constante com ela.	No contexto do Sistema, Pedido pode ter uma dependência de Pagamento , pois o pedido usa o pagamento em determinado momento, mas não necessariamente mantém uma relação contínua.
Agregação	Um tipo especial de associação que indica uma relação "tem um" onde uma classe contém ou agrupa outras classes sem controle total sobre seu ciclo de vida. A classe "contida" pode existir independentemente da classe que a "contém" .	No Sistema, Pedido pode ter uma agregação com Produto , indicando que um pedido "tem um" ou "possui" vários produtos, mas os produtos podem existir fora do contexto do pedido.
Composição	Um tipo mais forte de agregação, onde uma classe "contém" outra classe de forma que, se a classe "contente" for destruída, a classe contida também será. É uma relação de dependência completa, geralmente representada por um diamante preto.	Se Fornecedor tem uma composição com ContatoFornecedor , isso indica que os contatos de um fornecedor são específicos para ele e dependem de sua existência. Se o fornecedor for removido, os contatos relacionados também serão.
Generalização	Representa uma relação "é um" entre classes, onde uma classe filha herda atributos e métodos de uma classe pai. É indicado por uma seta com linha contínua que aponta para a classe pai.	Se Fornecedor fosse uma classe genérica e houvesse classes derivadas, como FornecedorLocal e FornecedorInternacional , essas classes específicas poderiam herdar as características comuns de Fornecedor com algumas especificidades.

Casos de Uso: Cenários

A técnica de caso de uso é amplamente utilizada pela comunidade de desenvolvimento de sistemas em todo o mundo. O modelo de caso de uso está especificado pela UML e é uma técnica para a descoberta de requisitos.

Os casos de uso consistem em descrever as interações entre os usuários e um sistema, podemos usar diversos formatos para escrever esta descrição, desde um texto narrativo informal, até um texto que descreve uma sequência de passos realizando o diálogo entre o ator e o sistema.

Podemos resumir que os casos de uso descrevem os cenários de utilização de um sistema por um ator. Um ator pode representar uma pessoa ou outros sistemas.

O diagrama de caso de uso é uma representação gráfica dos atores se relacionando com os casos de usos e pode expressa as relações entre os casos de uso. Na figura 7 são ilustrados os elementos de um diagrama de caso de uso.

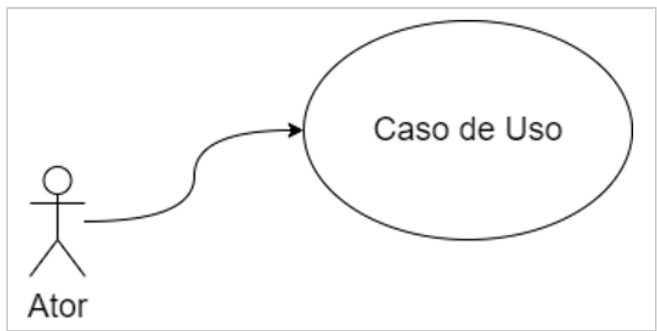


Figura 7: Elementos do Diagrama de Caso de Uso

Casos de Uso: Descrição

A descrição do caso de uso é essencial para o entendimento e discussão do cenário em questão. A descrição é a materialização das formas de interação do usuário com o sistema.

Veja a descrição a seguir: “Para eu agendar uma consulta no consultório médico, eu vou ligar e falar com a secretária, vou perguntar se o médico pode me atender no dia “d” e hora “h”, a secretária vai olhar na agenda do médico e me dizer que sim, então eu solicito que ela agende a consulta para mim. E ela vai me pedir alguns dados, como: meu nome, meu telefone, meu e-mail e qual o meu plano de saúde. E vai colocar meu nome na agenda.”

Observe que no texto ocorre a descrição de um cenário para de agendar uma consulta. Deste cenário ainda podemos ter um conjunto de outros cenários, como não ter o dia e hora disponível. Ou o médico não atender o plano de saúde.

Dessa descrição podemos listar uma série de requisitos, como; o médico tem uma agenda e essa agenda é organizada por dias e horas de atendimento; quem tem acesso a agenda é a secretária; o paciente tem acesso a agenda através da secretária; pode agendar, então também poderá cancelar; Depois de ocorrer a consulta pode-se marcar que a consulta foi realizada, então deve cobrar o plano de saúde.

A escrita no caso de uso pode ser formal, e utilizamos a seguinte estrutura do documento de caso de uso: nome do caso de uso; descrição; lista de atores; pré-condição; pós condição; fluxo básico de eventos; fluxos alternativos de eventos e fluxos de exceção.

História do Usuário (User Stories)

A história de usuário é uma técnica de descoberta de requisito muito utilizada e difundida em equipes de desenvolvimento que utilizam processos ágeis. Pois como os requisitos mudam rapidamente, não se gasta tempo com documentações formais.

A técnica consiste em organizar as histórias em cartões, e dessa forma a priorização dos requisitos ocorre com a discussão dos cartões e se define as prioridades.

Mas as histórias têm uma forma pré-estabelecida de escrita que segue o modelo: **Quem <usuário>, O quê <requisito>, Por quê <necessidade>**

Um outro modelo de escrita pode ser baseado no: **Como/Eu quero/Para**. Pode ser visualizado na figura 8.

Como _____ (tipo de usuário do produto)

Eu quero _____ (requisito)

Para _____ (necessidade a ser atendida)

Figura 8: Declaração: história de usuário (“Histórias de Usuário (User Stories)”, [s.d.])

A história do usuário deve conter apenas as informações necessárias para representar uma conversa entre os desenvolvedores e o Dono do Produto / [Product Owner \(PO\)](#)¹ sobre as necessidades e metas dos usuários.

A técnica de história do usuário é estruturada por 3 elementos, a saber: cartão, ser um pequeno espaço para escrever o suficiente; conversa, o requisito é compartilhado com a equipe via conversa; e confirmação, ter um método de validação com um critério de aceitação, representado na figura 9.

Dado que _____ (a pré-condição da ação)

Quando _____ (ação junto ao produto)

Então _____ (comportamento esperado do produto)

Figura 9: Critério de Aceitação (“Histórias de Usuário (User Stories)”, [s.d.])

As histórias de usuários devem passar pelo teste **INVEST** (***I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable*), o qual foi desenvolvido pelo especialista em desenvolvimento de *software* e métodos ágeis Bill Wake em 2003. O teste é uma ferramenta fundamental na gestão de projetos, em especial no refinamento do [Product Backlog Item \(PBI\)](#)², servindo como um *checklist* para garantir que cada PBI esteja bem definido, claro e pronto para ser implementado.

De modo a deixar o entendimento do teste INVEST mais claro, detalhamos a seguir o seu acrônimo - adaptado de (DIAS, 2022).

I - Independente: Cada PBI deve ser independente dos outros, evitando dependências que possam complicar a implementação e o teste. Quando os PBIs são fortemente dependentes, pode ser possível combiná-los em um único PBI.

N - Negociável: O escopo de um PBI pode ser negociado, mas a sua essência deve permanecer clara. Enquanto o PBI está no backlog do produto, ele pode ser reescrito ou até mesmo descartado, dependendo do negócio, mercado, técnico ou qualquer outro tipo de requisito dos membros da equipe.

V - Valioso: O PBI deve entregar valor ao cliente ou ao negócio. O PBI deve agregar valor às partes interessadas (*stakeholders*).

E - Estimável: É necessário ser possível estimar os esforços para implementar o PBI. Se o PBI não puder ser estimado, não fará sentido mantê-lo.

S - Small: O PBI deve ser **pequeno** (*small*) o suficiente para ser implementado em um único [Sprint](#)³. Uma boa regra geral é que qualquer Item de Backlog do Produto não leve mais de 50% de uma iteração. PBIs grandes geralmente são nomeados como "*Epics*", os quais levarão mais de uma iteração para ser entregues e, necessariamente, precisarão ser divididos em PBIs menores que possam caber confortavelmente em iterações menores.

T - Testável: Existem critérios claros de aceitação que permitem verificar se o PBI foi concluído com sucesso. Um PBI só deve ser considerado concluído, entre outras coisas, se for testado com sucesso. Se não for possível testar um PBI devido à falta de informações ou acesso é necessário avaliar se o mesmo é de fato "*Estimável*".

Curiosidade

Bill Wake é autor de diversos artigos e livros sobre desenvolvimento ágil e práticas de programação. Ele contribuiu para difundir e refinar práticas de desenvolvimento orientadas ao XP (*eXtreme Programming*) e é reconhecido por sua

habilidade em simplificar e tornar práticas ágeis mais acessíveis a desenvolvedores e equipes de projeto.

Para conhecer mais sobre o importante trabalho de Wake, você pode acessar diversos de seus artigos em <<https://xp123.com/author/bill/>>.

¹ Literalmente é a “**voz do cliente**” dentro do projeto, sendo responsável por maximizar o valor do produto final, traduzindo as necessidades dos usuários e do negócio (sob a ótica do usuário e cliente final) em um conjunto de requisitos claros e priorizados

² É a menor unidade de trabalho gerenciável.

³ Eventos que possuem duração fixa (períodos de tempo previamente definidos), os quais se iniciam imediatamente após a conclusão do anterior.

Modelagem de Processos

Um **processo de negócio** é uma sequência estruturada de atividades ou tarefas realizadas por pessoas para alcançar um objetivo específico dentro de uma organização. Esses processos representam o fluxo de trabalho e a interação entre pessoas, sistemas de informações, garantindo que as operações da organização sejam executadas de maneira eficiente e consistente. A compreensão e modelagem de processos de negócio são fundamentais para identificar oportunidades de melhoria, otimizar recursos e alinhar as atividades da organização com suas metas estratégicas. Em última análise, um processo de negócio bem definido permite que a organização entregue valor aos clientes e responda de maneira ágil às demandas do mercado.

Dando continuidade ao conceito de processos de negócio, é importante destacar que esses processos podem ser classificados em diferentes tipos, como operacionais, gerenciais e de suporte. **Processos operacionais** são aqueles diretamente envolvidos na entrega de valor ao cliente, como produção e atendimento ao cliente. Já os **processos gerenciais** apoiam a governança e a estratégia da organização, enquanto os **processos de suporte** fornecem os recursos necessários para que os processos operacionais possam ser executados, como o gerenciamento de TI e recursos humanos.

A **modelagem de processos de negócio** desempenha um papel fundamental na análise e otimização desses processos. Ela permite visualizar o fluxo de atividades, identificar gargalos e redundâncias, e entender as interações entre diferentes áreas da organização. Utilizando ferramentas de modelagem, como *Business Process Model and Notation* (BPMN), é possível documentar processos complexos de forma padronizada, facilitando a comunicação entre as equipes e promovendo a melhoria contínua.

Além disso, a **Gestão de Processos de Negócio** / *Business Process Management* (BPM) é uma abordagem que visa melhorar continuamente esses processos, promovendo eficiência e adaptabilidade. A BPM utiliza a modelagem de processos como base para automatizar, monitorar e aprimorar o desempenho organizacional, garantindo que a empresa se mantenha competitiva e alinhada às necessidades do mercado.

Em suma, processos de negócio bem modelados e gerenciados ajudam a maximizar a eficiência organizacional, reduzindo custos e aumentando a qualidade dos serviços e produtos oferecidos. Esse alinhamento entre processos e objetivos organizacionais é essencial para que as empresas possam entregar valor e se destacar em um ambiente de mercado dinâmico e competitivo.

A atividade de modelagem de processo de negócio como atividade da elicitação de requisitos segue os seguintes passos:

1. Definir o escopo do processo;

2. Identificar os *stakeholders* e responsáveis pelas atividades;
3. Mapear o fluxo de trabalho
4. Identificar e modelar as atividades e as decisões no fluxo;
5. Definir as entradas, as saídas e o fluxo de dados;
6. Identificar padrões e as regras de negócio; e
7. Validar o modelo de processo com os envolvidos.

A ferramenta que utilizamos como padrão para modelagem de negócios é BPMN, onde a especificação pode ser encontrada em <<https://www.bpmn.org/>>, na figura 10 temos um exemplo de modelagem de processo.

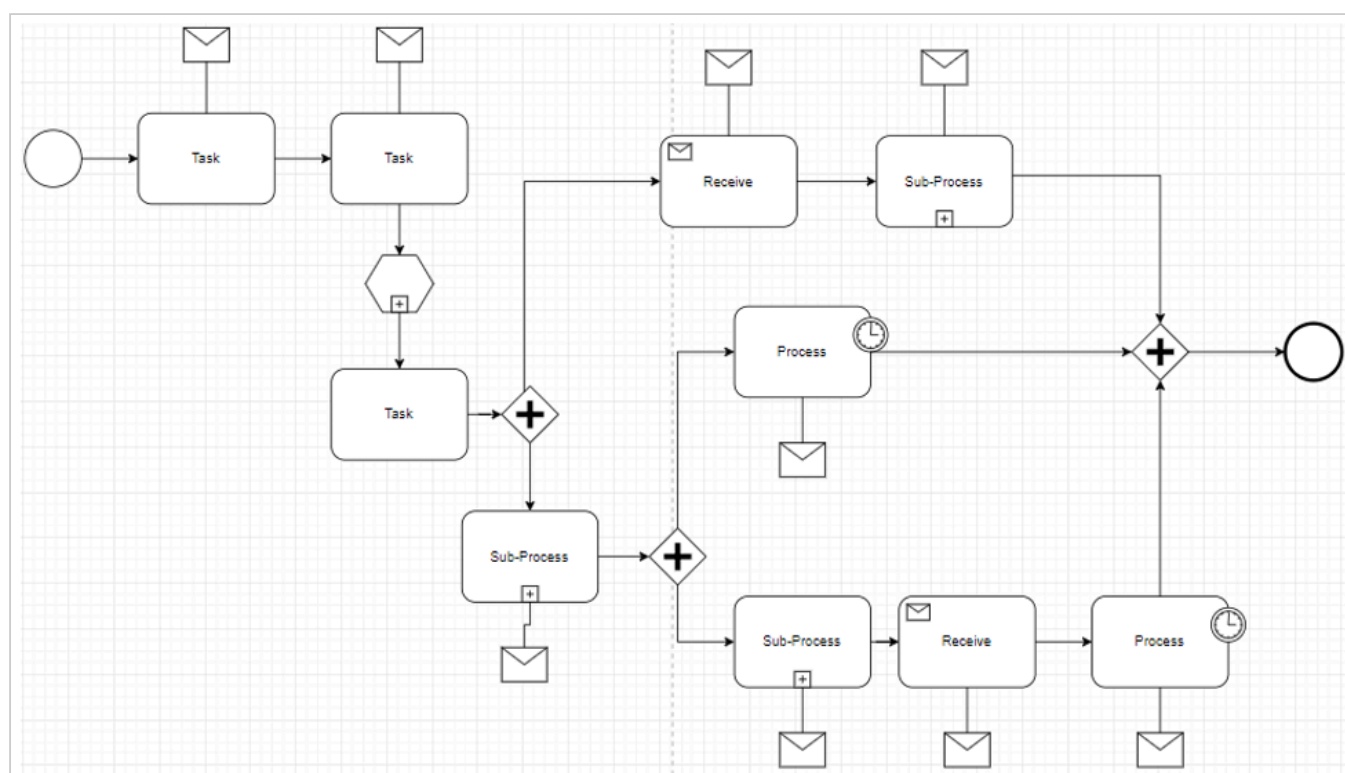


Figura 10: Exemplo de BPMN construído com o *LucidChart*® <<https://www.lucidchart.com/>> da empresa Lucid Software Inc.

Dicas

Várias soluções além do *LucidChart*® podem ser utilizadas para facilitar o desenho de processos de negócios. Dentre as várias opções possíveis, destacamos as que se seguem:

O projeto **BPMN.IO** <<https://bpmn.io/>> plataforma para criação de diagramas BPMN, mantida pela criadores da plataforma *Camunda*.

O **Bizagi Modeler** <<https://www.bizagi.com/pt/plataforma/modeler>> software gratuito de mapeamento e modelagem de processos de negócio; e a

Camunda <<https://camunda.com/platform/>> plataforma de automação orquestrada inteligente, a qual possui ainda um excelente conjunto de materiais instrucionais, com vídeos tutoriais e exemplos disponível em sua academia de aprendizagem "*Camunda Academy*" disponível em <<https://academy.camunda.com/>>.

IV - ANÁLISE DE REQUISITOS

Análise de Requisitos

A fase de análise é fundamental no processo de engenharia de requisitos, pois é aqui que se constrói o entendimento detalhado das necessidades dos *stakeholders*. Nessa etapa, os requisitos identificados são organizados, classificados, priorizados e negociados, garantindo que todos os envolvidos no projeto tenham uma visão compartilhada e estruturada sobre o que será desenvolvido.

A análise começa pela organização dos requisitos, uma etapa em que os *stakeholders* são agrupados de acordo com suas perspectivas e expectativas em relação ao sistema. Esse agrupamento permite criar uma estrutura inicial que facilita a visualização dos diferentes pontos de vista, proporcionando uma base sólida para a classificação dos requisitos. Classificar os requisitos é essencial para identificar diferentes categorias, como funcionalidades principais, requisitos técnicos e requisitos de segurança, estabelecendo uma hierarquia que facilita a priorização.

A priorização dos requisitos possibilita que o projeto concentre recursos e esforços nas entregas mais críticas e de maior valor para os *stakeholders*. Já a negociação é um aspecto contínuo e estratégico da análise, pois permite alinhar as expectativas e viabilidades, estabelecendo acordos claros e sustentáveis entre as partes envolvidas.

Durante a análise, são designados atributos relevantes aos requisitos, como prioridade, complexidade e risco, que fornecem informações essenciais para a tomada de decisões ao longo do desenvolvimento. Com a análise bem executada, a equipe de projeto consegue uma visão integrada e detalhada, o que facilita o planejamento e reduz riscos, aumentando as chances de sucesso na entrega do produto.

Análise: Classificação e Organização

A análise de requisitos é uma fase essencial para melhorar o entendimento das necessidades dos *stakeholders*. Nela são realizadas atividades de classificação, organização, priorização e negociação dos requisitos. Essa fase visa garantir que todos os envolvidos no projeto compartilhem uma visão clara dos objetivos, facilitando o alinhamento entre as expectativas dos *stakeholders* e as possibilidades técnicas e de negócio.

Uma abordagem eficaz para iniciar a análise é organizar os requisitos de acordo com os diferentes pontos de vista dos *stakeholders*. Agrupar os *stakeholders* com perspectivas semelhantes permite criar agrupamentos de requisitos alinhados a essas visões, facilitando a visualização das necessidades específicas de cada grupo e criando uma base sólida para o processo de classificação.

Com os requisitos organizados, o próximo passo é realizar uma classificação estruturada, que os divide em categorias distintas, como funcionais e não funcionais. Essa classificação facilita a priorização, ajudando a equipe de desenvolvimento a identificar quais requisitos são críticos para o sucesso do sistema e que devem ser abordados com prioridade. Esse processo contribui para uma compreensão completa e bem-organizada dos mesmos, garantindo que os esforços de desenvolvimento estejam alinhados com os objetivos do projeto.

A negociação é uma atividade central na análise de requisitos, pois é durante essa fase que ocorrem as discussões para alinhar os desejos dos *stakeholders* com as restrições do projeto. Por meio da negociação, a equipe pode definir compromissos claros sobre o que será desenvolvido, formando uma base sólida para o contrato de desenvolvimento do software.

O processo de análise envolve também a criação de uma lista detalhada dos requisitos identificados até o momento, com a atribuição de atributos essenciais para cada requisito, como:

- **Prioridade:** identifica a importância relativa de cada requisito.
- **Complexidade:** estima o nível de esforço e recursos necessários para a implementação.

- **Risco:** avalia os possíveis impactos que o requisito pode ter no projeto e no produto.

Essa abordagem completa de análise garante que os requisitos estejam bem definidos, organizados e alinhados com as expectativas, permitindo que o projeto avance com clareza e consistência nas próximas fases de desenvolvimento.

Análise de Viabilidade!

Após a elicitación dos requisitos, a atividade de análise de viabilidade é essencial para garantir que todos os requisitos descobertos possam ser implementados de maneira eficiente e eficaz. Essa análise ajuda a identificar potenciais problemas antes de entrar nas fases de design e implementação.

Na tabela 7 vamos destacar aspectos que devem ser analisados para garantir a correta realização do requisito em questão avaliado.

Tabela 7: Lista de Aspectos de Viabilidade de Requisitos

Aspecto	Descrição
Consistência e Coerência	Verifique se os requisitos não são conflitantes ou contraditórios entre si. Um conjunto de requisitos coerente garante que todos os <i>stakeholders</i> estejam alinhados e evita retrabalho.
Compleitude	Avalie se todos os requisitos necessários foram identificados e detalhados. Requisitos incompletos podem levar a lacunas no sistema final.
Viabilidade Técnica	Determine se a equipe de desenvolvimento possui as habilidades e os recursos técnicos necessários para implementar os requisitos especificados. Isso inclui a disponibilidade de ferramentas, tecnologias e infraestrutura
Viabilidade Econômica	Calcule os custos associados à implementação de cada requisito e compare-os com os benefícios esperados. Avalie se o orçamento disponível é suficiente para cobrir todos os custos previstos
Viabilidade Operacional	Verifique se os requisitos podem ser implementados dentro do ambiente operacional da organização. Isso inclui a compatibilidade com sistemas existentes e a capacidade dos usuários de adotar e utilizar o novo sistema
Riscos e Impactos	Identifique possíveis riscos associados à implementação de cada requisito e analise seus impactos potenciais. Desenvolva estratégias de mitigação para minimizar esses riscos
Priorização dos Requisitos	Classifique os requisitos com base em sua importância e urgência. Isso ajuda a focar nos requisitos críticos que devem ser atendidos primeiro

Priorização de Requisitos

A priorização de requisitos é fundamental para otimizar a alocação de recursos no desenvolvimento de sistemas, pois os recursos são limitados e exigem um uso estratégico. Assim, é essencial definir o que será desenvolvido primeiro e o que pode ser postergado. Em geral, os requisitos essenciais e necessários para o funcionamento básico do sistema devem receber a maior prioridade, garantindo que o projeto avance com o foco nas funcionalidades mais críticas.

Ao ajustar as prioridades de desenvolvimento, torna-se possível não só otimizar o uso dos recursos como também reduzir custos, já que os esforços estarão concentrados nas entregas de maior valor e impacto para o sistema.

Vamos apresentar os passos que devem ser seguidos para definir os critérios de priorização e indicação dos valores para auxiliar nas escolhas:

1. **Identificar os Critérios de Priorização:** Os critérios para priorizar os requisitos podem variar de acordo com o projeto, na tabela 8 são definidos o foco do critério e o que deve ser analisado.

Tabela 8: Critérios e Elementos de Análise

Critério	O que analisar
Impacto no Usuário Final	Qual o valor que o requisito agrega para o usuário?
Urgência para o Negócio	Qual a importância do requisito para os objetivos estratégicos do negócio?
Viabilidade Técnica	É tecnicamente possível implementá-lo com os recursos e tecnologias atuais?
Complexidade	Qual o nível de esforço necessário para implementar o requisito?

2. **Definir um Sistema de Intervalo de Valores para Priorização para atribuir valores numéricos a cada critério facilita a classificação. Um modelo com intervalo de valores de I a V é comum:**

I - **Baixa Prioridade:** Requisitos que não são críticos para a operação inicial e que podem ser implementados mais tarde.

II - **Média-Baixa Prioridade:** Requisitos úteis, mas que agregam valor apenas em cenários específicos ou complementares.

III - **Média Prioridade:** Requisitos importantes, mas não essenciais para o funcionamento básico do sistema. Ideal para a fase intermediária de desenvolvimento.

IV - **Média-Alta Prioridade:** Requisitos com valor estratégico significativo e impacto direto na experiência do usuário.

V - **Alta Prioridade:** Requisitos essenciais para o funcionamento e o sucesso do sistema, que devem ser implementados na primeira fase.
3. **Atribuir Pontuações aos Requisitos de forma que cada requisito é avaliado e recebe uma pontuação para cada critério, com base no sistema de intervalo de valores:** Por exemplo, um requisito essencial para o negócio e o usuário final pode receber pontuação **5** em impacto e urgência, mas uma pontuação **2** em complexidade devido a sua simplicidade de implementação.

4. **Calcular a Prioridade Geral das pontuações de cada critério, pode-se calcular uma pontuação total ou média para cada requisito:** Essa pontuação permite que os requisitos sejam classificados em ordem de prioridade. Um requisito com média alta, por exemplo, será priorizado sobre requisitos com médias mais baixas.

5. **Classificar e Visualizar os Requisitos a partir das pontuações, é possível classificar os requisitos em categorias como:**

- Essenciais:** Pontuação 4.5 - 5,0
- Importantes:** Pontuação 3.5 - 4,4
- Desejáveis:** Pontuação 2.5 - 3,4
- Baixa Prioridade:** Pontuação 1.0 - 2,4

Segue um exemplo de Priorização com Intervalos de Valores, na tabela 9. Considere uma lista de requisitos para um sistema de economia compartilhada em um condomínio:

Tabela 9: Tabela de Requisitos Priorizados

Requisito	Impacto	Urgência	Viabilidade Técnica	Complexidade	Prior
Cadastro de Condômino	5	5	4	3	4.25
Validação de Condômino	4	3	5	4	4.00
Agendamento de Objetos	5	4	4	3	4.00
Acesso por Ícones	3	2	5	2	3.00
Funcionalidade de Gamificação	2	2	4	3	2.75



Análise de Impacto

A análise de impacto é o processo de avaliar as consequências de mudanças nos requisitos de um projeto, e a **matriz de requisitos** é uma ferramenta essencial para isso, pois permite identificar rapidamente quais áreas do sistema e quais requisitos específicos serão afetados, facilitando o planejamento e a adaptação com base nas interdependências documentadas.

A matriz de requisitos, também conhecida como matriz de rastreabilidade de requisitos, é uma ferramenta essencial para documentar, organizar e gerenciar as demandas de um projeto. Ela permite que os requisitos identificados sejam rastreados ao longo do ciclo de vida do projeto, proporcionando visibilidade e controle sobre as especificações iniciais e suas respectivas evoluções.

À medida que os requisitos são levantados e analisados, eles são inseridos na matriz de requisitos, o que facilita a organização e o entendimento detalhado das necessidades do projeto. Essa documentação é fundamental para garantir o alinhamento entre os objetivos do projeto, as expectativas dos *stakeholders* e o escopo final do sistema, proporcionando um ponto de referência que ajuda a assegurar que o sistema entregue corresponda ao que o usuário final espera.

Em um projeto de sistema de economia compartilhada de objetos para um condomínio, por exemplo, a matriz de requisitos pode ser estruturada com campos como: código do requisito, prioridade, descrição, tipo, solicitante, status e data de conclusão. Esse formato permite uma visualização rápida e organizada das informações essenciais para cada requisito.

Na tabela 10 está uma matriz de requisitos exemplificada para um sistema de economia compartilhada de objetos em um condomínio:

Tabela 10: Matriz de Requisitos

Código	Prioridade	Descrição	Tipo	Solicitante	
1	Alta	Cadastro de condômino	Funcional	Equipe	C
2	Média	Validação de condômino	Segurança	Gerente	P
3	Alta	Cadastro de objeto	Funcional	Equipe	Em a
4	Média	Acesso por ícones	Usabilidade	Cliente	P
5	Alta	Agendamento de objeto	Funcional	Equipe	Em a

Essa matriz fornece uma visão clara do status e das prioridades dos requisitos, ajudando a equipe a acompanhar o progresso e a manter o foco nos itens de maior importância. Além disso, ela permite que todos os envolvidos no projeto tenham um entendimento compartilhado do escopo e das demandas, facilitando o alinhamento entre as áreas de negócios e desenvolvimento.

Documentação de Requisitos

Na fase de especificação, formalizamos os requisitos do sistema em um documento específico, transformando as ideias e entendimentos preliminares em descrições detalhadas e precisas. Durante as fases de elicitação e análise, os requisitos geralmente são organizados de maneira preliminar e iterativa, permitindo que sejam refinados conforme o entendimento das necessidades dos *stakeholders* se aprofunda. A especificação, portanto, é o momento de consolidar e formalizar os requisitos, detalhando-os de maneira clara e completa para servir de base para o contrato e guiar a execução do projeto.

A especificação dos requisitos pode ser estruturada de várias maneiras, adaptando-se às necessidades do projeto e aos padrões da organização. Algumas notações comuns incluem:

- **Linguagem natural:** uma lista de frases claras e concisas, onde cada sentença expressa um requisito específico.
- **Formulários estruturados:** documentos com campos que organizam as informações sobre cada requisito, como descrição, prioridade, dependências, entre outros.
- **Modelos gráficos usando UML (*Unified Modeling Language*):** diagramas que representam visualmente a estrutura e o comportamento do sistema, como casos de uso, diagramas de classes, entre outros.
- **Notação matemática:** uma especificação formal que utiliza expressões matemáticas para descrever requisitos, proporcionando precisão e clareza em contextos em que ambiguidade não é tolerada.

Cada metodologia de desenvolvimento adota seus próprios artefatos e formatos para a especificação dos requisitos. No mercado de trabalho, é comum que a empresa forneça diretrizes específicas para a criação e preenchimento desses documentos, adaptando-os à metodologia de desenvolvimento em uso, como por exemplo os modelos Ágil, em Cascata ou Híbrido.

De forma geral, um documento de requisitos pode incluir as seguintes seções:

Prefácio: contextualiza o documento e seu propósito.

Introdução: apresenta o sistema, seu escopo e objetivos gerais.

Glossário: define termos e abreviações importantes para o entendimento dos requisitos.

Requisitos do usuário: descreve as necessidades e expectativas dos usuários.

Arquitetura do sistema: fornece uma visão geral da estrutura do sistema.

Requisitos do sistema: detalha as funcionalidades e características que o sistema deve possuir.

Modelo do sistema: inclui diagramas e representações visuais dos requisitos.

Evolução do sistema: discute possíveis expansões e manutenção futuras.

Apêndices e índice: materiais adicionais que complementam a especificação.

Como apontado por (SOMMERVILLE, 2019), o detalhamento desses itens é fundamental para o sucesso do projeto. Mesmo com especificações minuciosas, é importante lembrar que os requisitos estão sujeitos a mudanças. Portanto manter a documentação bem-organizada permite que adaptações e manutenções futuras sejam realizadas de forma controlada e eficiente.

Saiba mais

Para conhecer sobre o funcionamento do *Jira*, não deixe de assistir ao excelente tutorial do canal da "*Academy for International Modern Studies (AIMS)*", intitulado "***Agile Waterfall Hybrid Model? Difference between Agile and Waterfall & How to Combine?***" em tradução livre algo como "**Modelo Ágil Híbrido em Cascata? Diferença entre Ágil e Em Cascata e Como Combiná-los?**", disponível no YouTube em: <<https://youtu.be/osk01i4OTBA>>

Como já pudemos perceber o melhor modelo de desenvolvimento a ser seguido é aquele (ou aqueles) que melhor se adapta à nossa realidade e necessidade, sendo assim lhe propomos o desafio de pesquisar e conhecer mais alguns modelos. Para tal sugerimos os modelos tradicionais "*Espiral, em V e Orientado a Reuso*" - e os modelos ágeis "*Scrum, Kanban e Lean*".

V - ESPECIFICAÇÃO DE REQUISITOS

Especificação de Requisitos de Software (SRS)

A Especificação de Requisitos de Software / *Software Requirements Specification* (SRS) é um documento que descreve detalhadamente todos os requisitos funcionais e não funcionais de um sistema. Baseado no padrão IEEE 830 ("IEEE Recommended Practice for Software Requirements Specifications", 1994), o SRS é fundamental para assegurar uma comunicação clara entre todos os envolvidos no projeto, desde desenvolvedores até partes externas, como usuários e *stakeholders*.

O propósito do SRS é reunir e organizar todos os requisitos do projeto de forma estruturada, facilitando o entendimento do que o sistema deve realizar para atender às necessidades dos usuários. Esse documento pode ser composto por um conjunto único ou por diversos artefatos e ferramentas que detalham os requisitos, tais como casos de uso, modelos de casos de uso, especificações suplementares, entre outros.

Mais do que um simples registro, o SRS é um artefato dinâmico e ativo, que pode ser atualizado à medida que novos recursos são adicionados ou modificados ao longo do desenvolvimento. Ele serve como um referencial para o projeto, auxiliando tanto na fase de design quanto na de implementação. Além disso, é uma ferramenta essencial para as equipes de teste e garantia de qualidade, pois proporciona uma base para a criação de casos de teste que assegurem que o sistema cumpre os requisitos estabelecidos.

O SRS também desempenha um papel fundamental para o Gerente de Projetos, que o utiliza como padrão de referência para monitorar o andamento do desenvolvimento e verificar a aderência ao escopo planejado. Ao final, o SRS representa um contrato informal entre todas as partes, garantindo que o produto final estará alinhado com as especificações acordadas.

Padrões e Templates (Modelos) de Documentação

No desenvolvimento de software, a utilização de padrões e templates de documentação é essencial para garantir clareza, consistência e eficiência no processo de registro e comunicação dos requisitos. O [Rational Unified Process \(RUP\)](#)¹, uma metodologia amplamente adotada, especifica o uso de templates padronizados para auxiliar na documentação dos requisitos de software, sendo um dos mais importantes o template da Especificação de Requisitos de Software (SRS).

Esse documento, estruturado de acordo com normas como o IEEE 830, proporciona uma forma organizada de descrever tanto os requisitos funcionais quanto os requisitos não funcionais do sistema. A padronização do SRS facilita a comunicação entre as equipes e outras partes interessadas, garantindo que todos compreendam claramente o escopo e as funcionalidades esperadas do sistema, além de servir como uma base sólida para o design, a implementação e a verificação do software.

A adoção de padrões e templates de documentação, como o SRS especificado pelo RUP contribui para um desenvolvimento de software mais estruturado e controlado, reduzindo riscos e promovendo uma melhor gestão de requisitos ao longo do ciclo de vida do projeto. Na figura 11 destacamos o sumário do documento modelo de especificação de requisitos SRS.

¹ Para conhecer em detalhes a metodologia RUP, acesse o excelente material desenvolvido pela IBM, intitulado "**Tutorial: Orientação sobre o Rational Unified Process**" disponível em: <<https://www.ibm.com/docs/pt-br/rsas/7.5.0?topic=learn-tutorial-rational-unified-process-guidance>>

funcionais e não funcionais. Adotar uma estrutura consistente e utilizar modelos visuais, como diagramas de caso de uso, também são passos fundamentais para criar uma documentação que seja fácil de navegar e compreender.

Além disso, revisar e validar a documentação com *stakeholders* e mantê-la atualizada ao longo do desenvolvimento são ações cruciais para que os requisitos reflitam as necessidades reais do projeto. Essas práticas asseguram que a documentação de requisitos sirva como uma base sólida e confiável, apoiando o sucesso do desenvolvimento e a entrega de um software que realmente atende às expectativas dos usuários.

Vamos listar algumas das melhores práticas para garantir uma documentação clara e concisa de requisitos de software: **1)** use linguagem simples e objetiva; **2)** utilize especificidade e precisão para evitar ambiguidades; **3)** estrutura consistente e adoção de padrões de formatação; **4)** evitar redundâncias; **5)** revisar e validar com os *stakeholders*; **6)** documentação iterativa e adaptável; e **7)** revise e verifique regularmente.

VI - VALIDAÇÃO E GERENCIAMENTO DE REQUISITOS

Validação e Gerenciamento de Requisitos

Na fase de validação de requisitos, o objetivo é verificar se os requisitos especificados realmente atendem às necessidades e expectativas dos usuários. Esta etapa envolve uma análise rigorosa que, ao identificar possíveis defeitos, promove a correção antecipada e assegura a conformidade com os objetivos do projeto. A validação, portanto, é uma oportunidade de detectar e corrigir problemas antes de avançar para as fases posteriores, onde os custos de correção aumentam significativamente.

No modelo ADIT, cada fase do processo — Análise, Design, Implementação e Teste — tem impacto nos custos de correção. Se um defeito passa despercebido da fase de análise para a fase de design, seu custo de correção aumenta, e essa escalada continua nas fases subsequentes. Assim, a validação na fase inicial é essencial para mitigar esses custos e aumentar a qualidade do projeto. Sommerville (2019), propõe cinco tipos de conferências essenciais durante a validação:

- 1. Conferência da validade:** verifica se os requisitos refletem as necessidades reais dos usuários, evitando a implementação de funcionalidades desnecessárias ou irrelevantes.
- 2. Conferência da consistência:** assegura que não existam conflitos entre os requisitos, garantindo que todos os itens especificados no documento estejam em harmonia.
- 3. Conferência da completude:** confirma que o documento de requisitos cobre todas as funcionalidades e restrições desejadas pelos usuários.
- 4. Conferência do realismo:** avalia a viabilidade dos requisitos, verificando se podem ser implementados dentro das limitações tecnológicas, de orçamento e de cronograma.
- 5. Verificabilidade:** garante que os requisitos sejam testáveis, reduzindo potenciais conflitos entre cliente e contratante ao longo do desenvolvimento.

Para realizar uma validação eficaz, diferentes técnicas podem ser aplicadas, como:

- Revisões sistemáticas:** equipes de revisores examinam o documento de requisitos para identificar inconsistências, ambiguidades e omissões.
- Prototipação:** criação de modelos executáveis do sistema para avaliar a funcionalidade e obter feedback antecipado dos *stakeholders*.
- Casos de teste:** desenvolvimento de cenários de teste para verificar se os requisitos são implementáveis e alinhados com as expectativas do usuário.

A fase de validação, portanto, é um passo crítico para assegurar a qualidade e a conformidade dos requisitos antes de avançar para o design e implementação. Com uma validação bem conduzida, o projeto reduz os riscos de retrabalho e aumenta as chances de sucesso na entrega final.

Controle de Mudanças

A documentação de requisitos de software desempenha um papel fundamental no desenvolvimento de sistemas eficazes, servindo como um guia claro e detalhado sobre o que o sistema deve realizar para atender às necessidades dos usuários e *stakeholders*. No entanto, para que essa documentação seja realmente útil, ela precisa ser clara, concisa e organizada de forma que facilite a compreensão e a comunicação entre todos os envolvidos no projeto. É aqui que as melhores práticas entram em cena.

Seguir melhores práticas para a documentação de requisitos de software ajuda a minimizar ambiguidades, reduzir mal-entendidos e garantir que todos compartilhem uma visão comum do projeto. Entre essas práticas, destacam-se o uso de uma linguagem simples e objetiva, a especificação precisa de cada requisito, e a separação entre requisitos funcionais e não funcionais. Adotar uma estrutura consistente e utilizar modelos visuais, como diagramas de caso de uso, também são passos fundamentais para criar uma documentação que seja fácil de navegar e compreender.

Além disso, revisar e validar a documentação com *stakeholders* e mantê-la atualizada ao longo do desenvolvimento são ações cruciais para que os requisitos reflitam as necessidades reais do projeto. Essas práticas asseguram que a documentação de requisitos sirva como uma base sólida e confiável, apoiando o sucesso do desenvolvimento e a entrega de um software que realmente atende às expectativas dos usuários.

O controle de mudança de requisitos na engenharia de requisitos assegura que o desenvolvimento de software permaneça alinhado com os objetivos de negócios e com as necessidades reais dos usuários, enquanto minimiza riscos e mantém o projeto em uma trajetória previsível e controlada.

Rastreabilidade

O processo de rastreabilidade de requisitos é uma prática essencial na engenharia de requisitos que permite acompanhar o ciclo de vida de um requisito, desde a sua origem como uma necessidade do usuário até a sua implementação em código. Essa rastreabilidade garante que todos os requisitos definidos no início do projeto sejam atendidos no produto final, facilitando a verificação de que o sistema desenvolvido atende às expectativas dos *stakeholders*. Esse processo envolve três etapas principais:

1. Rastreabilidade para Frente (*Forward Traceability*)

- A rastreabilidade para frente começa com a identificação da necessidade do usuário e acompanha o caminho até os requisitos de sistema, passando pelo design, pela implementação e, finalmente, pelo código. Cada requisito funcional e não funcional é rastreado, garantindo que ele seja devidamente traduzido em especificações de design e em funcionalidades no código.
- Esse processo é essencial para assegurar que as necessidades dos usuários estão sendo refletidas no desenvolvimento e que cada requisito inicial está sendo abordado na implementação.

2. Rastreabilidade para Trás (*Backward Traceability*)

- A rastreabilidade para trás permite mapear elementos do código e do design de volta para os requisitos originais. Se um desenvolvedor está implementando uma funcionalidade ou revisando uma parte do sistema, ele pode verificar quais requisitos estão relacionados a essa funcionalidade.

- Isso ajuda a evitar a introdução de funcionalidades desnecessárias ou não solicitadas (também chamadas de “*gold-plating*”), além de garantir que todos os elementos implementados tenham uma justificativa ligada aos requisitos iniciais.

3. Rastreabilidade Bidirecional (*Bidirectional Traceability*)

- A rastreabilidade bidirecional integra a rastreabilidade para frente e para trás, permitindo acompanhar os requisitos tanto de sua origem até o código quanto do código de volta até os requisitos. Isso é fundamental para uma visão completa de como as necessidades dos usuários são traduzidas em funcionalidades e para verificar a cobertura de todos os requisitos.
- Em termos práticos, isso facilita responder a perguntas como “*este requisito foi implementado?*” e “*esta funcionalidade no código atende a qual requisito?*”. Essa visibilidade total é crucial para auditorias, revisões e validação do sistema.

Para manter a rastreabilidade de requisitos, é comum usar ferramentas de gerenciamento de requisitos, como: *Jira* <<https://www.atlassian.com/br/software/jira>>, *Trello* <<https://trello.com/pt-BR>> ou *Microsoft Azure DevOps* <<https://azure.microsoft.com/pt-br/products/devops>>, que permitem criar links entre requisitos, casos de uso, documentos de design, tarefas de desenvolvimento e código. Cada requisito é identificado com um código único ou ID que facilita seu rastreamento em toda a documentação do projeto.

A rastreabilidade de requisitos, quando realizada corretamente, traz os seguintes benefícios:

- **Conformidade e Auditoria:** Em projetos que exigem conformidade com regulamentações, como o setor de saúde ou finanças, a rastreabilidade é vital para garantir que todos os requisitos regulamentares foram atendidos.
- **Facilidade de Gerenciamento de Mudanças:** Com a rastreabilidade, ao alterar um requisito, é possível identificar facilmente os elementos de design e código afetados, permitindo um gerenciamento mais eficiente de mudanças.
- **Qualidade e Testes:** A rastreabilidade facilita a criação de casos de teste para cada requisito e ajuda a verificar que o sistema final atende completamente aos requisitos definidos.

O processo de rastreabilidade de requisitos proporciona uma linha de conexão entre as necessidades dos usuários e o produto final, oferecendo uma visão clara de como as expectativas dos *stakeholders* são realizadas no sistema e permitindo o controle de mudanças e validação de qualidade.

Ferramentas de Gerenciamento de Requisitos (*Jira, Trello*)

É fundamental reconhecer que a eficácia na gestão de requisitos é vital para o sucesso de qualquer projeto de software. Ferramentas como o *Jira* e o *Trello* desempenham papéis significativos nesse contexto, cada uma com características que atendem a diferentes necessidades das equipes.

O *Jira* é amplamente utilizado por equipes de desenvolvimento de software que adotam metodologias ágeis. Ele oferece recursos avançados para planejamento de *Sprints*, rastreamento de bugs e gerenciamento de tarefas, permitindo uma rastreabilidade detalhada dos requisitos desde a concepção até a implementação. Sua capacidade de integração com outras ferramentas e a personalização de fluxos de trabalho o tornam uma escolha robusta para projetos complexos que exigem um controle rigoroso dos requisitos e das mudanças associadas. (ATLASSIAN, [s.d.])

“O *Jira* é a ferramenta nº 1 de gestão ágil de projetos usada pelas equipes para planejar, monitorar, lançar e dar suporte a softwares de alto nível com confiança. É a fonte única de informações de todo o ciclo de vida de desenvolvimento, permitindo que as equipes autônomas tenham o contexto necessário para que ajam com rapidez e

permaneçam alinhadas à meta de negócios mais importante. Seja para gerenciar projetos simples ou impulsionar as práticas de [DevOps¹](#), com o Jira as equipes avançam no trabalho, permanecem alinhadas e se comunicam sobre o contexto com mais facilidade.”

(ATLASSIAN, [s.d.])

¹ Junção de "Desenvolvimento" e "Operações" - são metodologias, processos e ferramentas que promovem a colaboração entre as equipes de desenvolvimento de software (*Dev*) e operações de TI (*Ops*). O objetivo dessas práticas é acelerar a entrega de software, melhorar a qualidade e confiabilidade dos sistemas, e facilitar a adaptação a mudanças e novas demandas.

Saiba mais

Para conhecer sobre o funcionamento do *Jira*, não deixe de assistir ao excelente tutorial do canal “*Eric Grassi*”, intitulado “**Como Usar o JIRA? TUTORIAL COMPLETO do Jira Software para Gestão de Projetos**” disponível no YouTube em:

<https://youtu.be/Ymf5fUkP_rE>

Por outro lado, o *Trello* é conhecido por sua interface visual intuitiva baseada em quadros e cartões, facilitando a organização e o acompanhamento de tarefas de forma simples e colaborativa. Embora não ofereça a mesma profundidade de recursos do *Jira*, o *Trello* é ideal para equipes que buscam uma solução leve e flexível para gerenciar requisitos e tarefas, especialmente em projetos menores ou menos complexos. Na figura 12 exibimos uma imagem da ferramenta *Trello*.

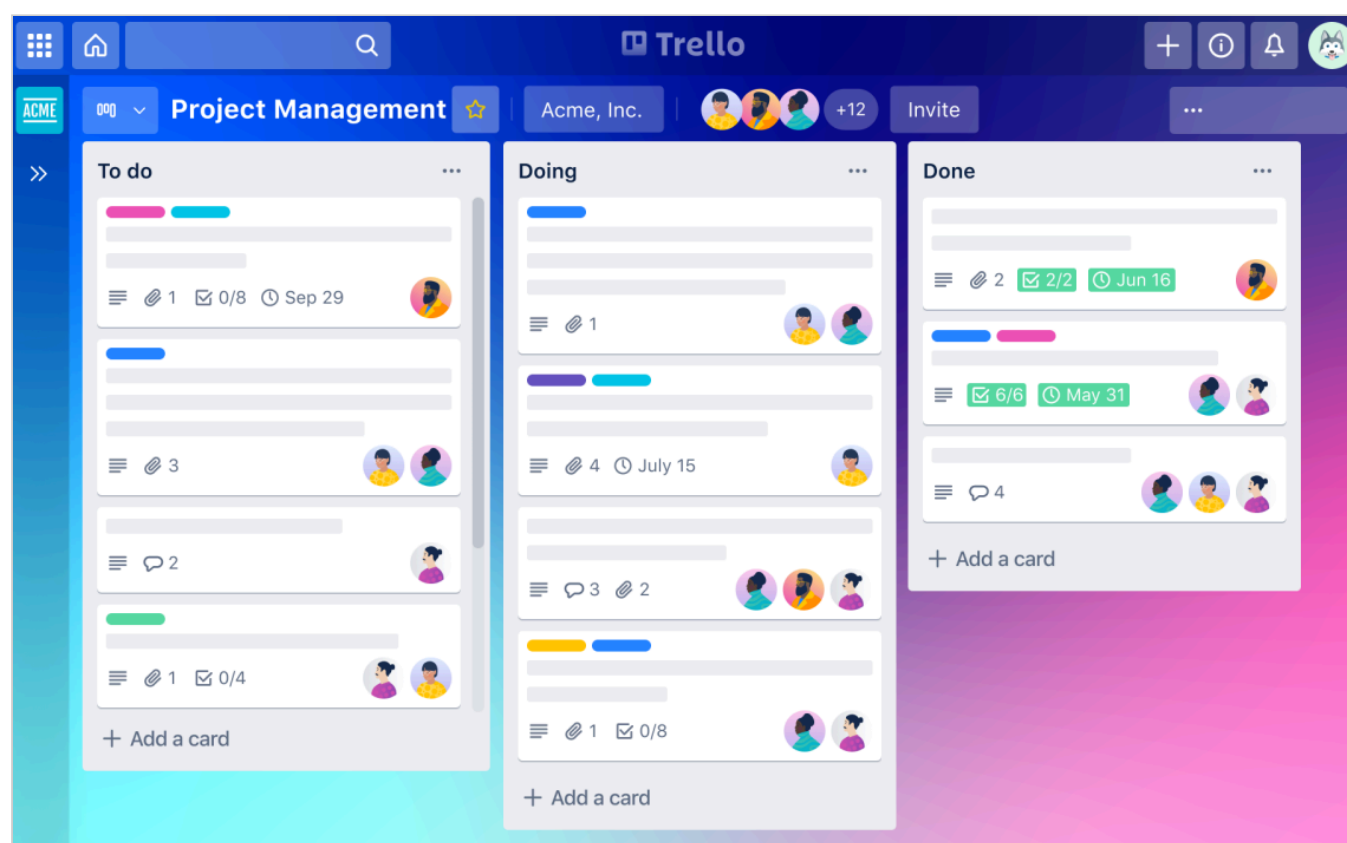


Figura 12: Exemplo de Tela da Ferramenta de Gerenciamento de Projetos do *Trello* <<https://trello.com/use-cases/project-management>>

A escolha entre essas ferramentas deve considerar as especificidades do projeto, a complexidade dos requisitos e a metodologia adotada pela equipe. Independentemente da ferramenta selecionada, o uso adequado de sistemas de gerenciamento de requisitos contribui para uma comunicação clara entre os *stakeholders*, facilita o controle de mudanças e assegura que o produto final atenda às expectativas estabelecidas.

Em suma, o gerenciamento eficaz de requisitos, apoiado por ferramentas adequadas como o *Jira* e o *Trello*, é um componente essencial para o desenvolvimento de software de qualidade, alinhado às necessidades dos usuários e aos objetivos do negócio.

Saiba mais

Para conhecer sobre os primeiros passos sobre o funcionamento do *Trello*, não deixe de assistir ao tutorial intitulado “**Tutorial completo de Trello 2024 - Gestão de tarefas pessoais e da equipe**” do canal “*Bizz-up*”, disponível no YouTube em: <<https://youtu.be/yEjI1KLIYQs>>

Até já, estudante!

Estamos chegando ao final do nosso curso sobre Análise de Requisitos, e que jornada enriquecedora tivemos! Exploramos juntos as etapas e as melhores práticas para coletar, documentar e gerenciar requisitos, essenciais para qualquer projeto de desenvolvimento de software. Ao longo desse percurso, ficou evidente a importância de compreender e organizar os requisitos de forma clara e eficiente, permitindo que a tecnologia responda às reais necessidades dos usuários e do mercado.

Com o conhecimento que você adquiriu, esperamos que esteja preparado(a) para enfrentar os desafios do desenvolvimento de sistemas com uma abordagem estruturada e estratégica. Você agora compreende como transformar as demandas dos *stakeholders* em um conjunto sólido de requisitos, contribuindo para a entrega de soluções que realmente fazem a diferença.

Como próximos passos, recomendamos que você explore tópicos relacionados à *gestão de dados* e aos *tipos de dados*. A gestão de dados é fundamental para garantir a qualidade, segurança e acessibilidade das informações em sistemas modernos, especialmente em projetos que envolvem grandes volumes de dados e requisitos complexos. Entender os tipos de dados, suas características e usos específicos também permitirá que você escolha abordagens e soluções mais adequadas para diferentes contextos de projeto.

Ao prosseguir seus estudos e projetos, lembre-se: a tecnologia é uma ferramenta poderosa quando usada com propósito e precisão. Esperamos que você leve consigo o entusiasmo e o comprometimento para aplicar o que aprendeu e continue evoluindo nessa área.

Parabéns por essa conquista, e até logo! Que essa seja apenas uma das muitas etapas em sua jornada de sucesso no mundo da tecnologia.

Vá em frente, sempre!

Tags do conteúdo

Análise de Requisitos, Engenharia de Requisitos, ADIT, NM (Necessidades e Metas), ICONIX, INVEST

Stakeholders, PBI, Product Backlog Item, Scrum, Sprint, RUP, Rational Unified Process

Tipos de Requisitos, Especificação dos Requisitos, Especificação de Requisitos de Software (SRS)

Ciclo de Vida, Elicitação, Unified Modeling Language, UML, Casos de Uso, User Stories, Ator, Cenário, Diagrama de Classe

Entrevistas, Workshops, Questionários, Observação, Prototipagem, Grupo Focal, Focus Group, JAD, Joint Application Development

Modelagem de Processos, Validação, Gerenciamento de Requisitos

Business Process Model and Notation, BPMN, Gestão de Processos de Negócio, Business Process Management, BPM

Análise de Viabilidade, Priorização de Requisitos Análise de Impacto

Padrões de Documentação, Templates de Documentação, Práticas de Documentação

Controle de Mudanças, Rastreabilidade, Ferramentas de Gerenciamento de Requisitos

JIRA, Trello, LucidChart, BPMN.IO, Camunda, Bizagi

Referências

ATLASSIAN. **Jira vs Trello**. Disponível em: <<https://www.atlassian.com/br/software/jira/comparison/jira-vs-trello>>. Acesso em: 06 out. 2024.

_____. **Comece a usar o Jira: guia completo para iniciantes**. Disponível em: <<https://www.atlassian.com/br/software/jira/guides/getting-started/introduction>>. Acesso em: 05 out. 2024.

BPMN.IO. **Web-based tooling for BPMN, DMN, CMMN and Forms**. Disponível em: <<https://bpmn.io/>>. Acesso em: 12 out. 2024.

Camunda Academy. Disponível em: <<https://academy.camunda.com/>>. Acesso em: 02 out. 2024.

DIAS, J. P. **O INVEST na Criação de Softwares**. Disponível em: <<https://www.linkedin.com/pulse/o-invest-na-cria%C3%A7%C3%A3o-de-softwares-jo%C3%A3o-paulo-dias-pmp/>>. Acesso em: 14 out. 2024.

Histórias de Usuário (User Stories). Disponível em: <<https://pim-go.com/fase-de-amadurecimento-modelo-agil/user-story/>>. Acesso em: 11 out. 2024.

Iconix: Fases. Iconix, [s.d.]. Disponível em: <<https://siiconix.blogspot.com/p/fases.html>>. Acesso em: 09 out. 2024.

IEEE Recommended Practice for Software Requirements Specifications. IEEE Std 830-1993, p. 1-32, abr. 1994.

InfoSphere Data Architect 9.1.0. Disponível em: <<https://www.ibm.com/docs/en/ida/9.1?topic=artifacts-requirement-documents>>. Acesso em: 22 out. 2024.

O melhor software de BPMN online. Disponível em: <<https://www.lucidchart.com/pages/pt/exemplos/bpmn-online>>. Acesso em: 05 out. 2024.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021. 672 p.

SEBESTA, R. W. **Conceitos de linguagens de programação**. 11. ed. Porto Alegre: Bookman, 2018.

Software gratuito de mapeamento e modelagem de processos de negócio - Bizagi Modeler. Disponível em: <<http://www.bizagi.com/pt/plataforma/modeler>>. Acesso em: 29 set. 2024.

SOMMERVILLE, I. **Engenharia de Software**. Tradução: Luiz Cláudio Queiroz. 10ª ed. São Paulo: Pearson, 2019.

Técnicas para levantamento de Requisitos. Disponível em: <<https://www.devmedia.com.br/tecnicas-para-levantamento-de-requisitos/9151>>. Acesso em: 28 set. 2024.

The Universal Process Orchestrator. Disponível em: <<https://camunda.com/>>. Acesso em: 30 set. 2024.

Nota:

Todas as marcas citadas e/ou exibidas neste material, pertencem aos seus respectivos fabricantes e/ou desenvolvedores.